Aprendizaje por refuerzo I

Resumen teórico integrado de los TP1, TP2 y TP3

Monte Carlo off policy con muestreo por importancia en FrozenLake, SARSA y Q learning en CliffWalking, DQN y Dueling DQN en Enduro.

Basado en las diapositivas del curso (carpeta docs del repositorio) y en Sutton y Barto, Reinforcement Learning: An Introduction (2018), citado en su bibliografía.

Cómo leer este documento

Cada término que aparece en una ecuación tiene una nota numerada que explica qué es, para qué sirve, cómo se obtiene y cuánto puede valer en los TP. Las notas se escriben una sola vez, la primera vez que el término aparece; después, cada ecuación lista sus términos con el número de nota correspondiente. Las secciones van de lo general a lo particular: el marco común, Monte Carlo (TP1), diferencia temporal (TP2) y aproximación con redes (TP3). Cada una reutiliza lo anterior y solo agrega lo nuevo.

Todos los métodos del documento comparten una misma forma de actualización, que se presenta en la sección 2 y reaparece en las siguientes:

$$\text{estimación} \leftarrow \text{estimación} + \text{paso} \cdot (\text{objetivo} - \text{estimación})$$

Lo que cambia de un método a otro es qué se usa como objetivo y cuánto mide el paso.

Contenido

1. El marco común: proceso de decisión de Markov
2. Monte Carlo off policy con muestreo por importancia (TP1)
3. Diferencia temporal: SARSA y Q learning (TP2)
4. Aproximación con redes: DQN y Dueling DQN (TP3)
5. Mapa integrador e índice de notas

1. El marco común: proceso de decisión de Markov

Los tres TP resuelven el mismo problema con herramientas cada vez más generales: un agente que aprende a actuar por prueba y error, recibiendo recompensas de un entorno. El modelo matemático de esa interacción es el proceso de decisión de Markov, o MDP (Proceso de decisión de Markov, págs. 22 a 27). En cada paso el agente observa un estado, elige una acción y el entorno responde con una recompensa y un estado nuevo.

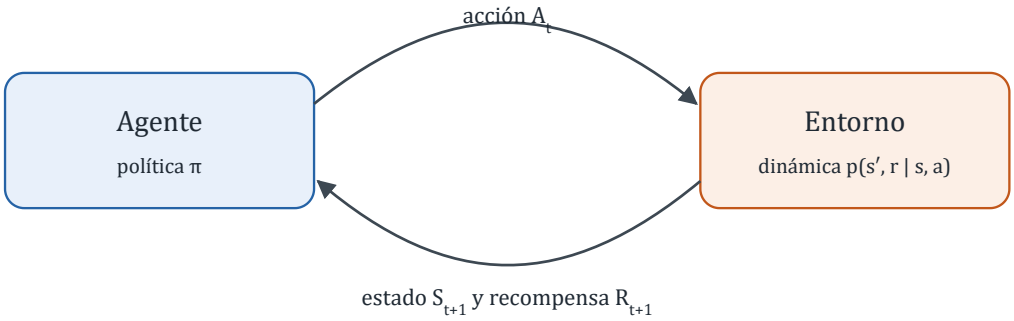


Figura 1. Ciclo agente entorno. El subíndice t+1 marca lo que llega después de actuar.

Nota 1 t, T paso de tiempo y fin del episodio

Qué es	El índice del paso dentro de un episodio. T es el paso en que el episodio termina. Un episodio es una partida completa, desde el estado inicial hasta un estado terminal.
Para qué sirve	Ordenar la experiencia. Con t se nombra el presente y con t+1 lo que viene después de actuar.
Cómo se obtiene	Un contador que arranca en 0 con env.reset() y suma 1 con cada env.step(). El episodio termina cuando el entorno devuelve terminated (meta, agujero, fin del juego) o truncated (tope de pasos).
Cuánto vale	Un entero entre 0 y T. En FrozenLake los episodios duran pocas decenas de pasos, con un tope de 100. En CliffWalking el camino más corto tiene 13. En Enduro cada paso del agente son 4 cuadros del juego y un episodio dura miles de pasos.

Nota 2 $S_t, s, \mathcal{S}$ estado

Qué es	La descripción de la situación en el paso t. $\mathcal{S}$ es el conjunto de todos los estados. Las mayúsculas indican la variable aleatoria (lo que ocurre en un paso) y las minúsculas un valor concreto.
Para qué sirve	Es lo único que el agente ve para decidir. Debe cumplir la propiedad de Markov: el futuro depende del estado actual y no de cómo se llegó a él (Proceso de decisión de Markov, pág. 13).
Cómo se obtiene	Lo devuelve el entorno en reset y en step.
Cuánto vale	En FrozenLake, un entero de 0 a 15 (fila · 4 + columna). En CliffWalking, de 0 a 47 (fila · 12 + columna). En Enduro, una pila de 4 imágenes de 84×84 con valores de 0 a 255: hay 256 elevado a 28224 estados posibles, y por eso una tabla es imposible (sección 4).

Nota 3 $A_t, a, |\mathcal{A}|$ acción

Qué es	La acción que el agente elige en el paso t. $\mathcal{A}$ es el conjunto de acciones posibles y $|\mathcal{A}|$ su cantidad.
Para qué sirve	Es la única forma que tiene el agente de influir en el entorno.
Cómo se obtiene	La elige la política a partir del estado.
Cuánto vale	En FrozenLake, 4 acciones: 0 ←, 1 ↓, 2 →, 3 ↑. En CliffWalking también 4, con otra codificación: 0 ↑, 1 →, 2 ↓, 3 ←. En Enduro, 9: acelerar, frenar, izquierda, derecha y sus combinaciones.

Nota 4 R_{t+1}, r recompensa

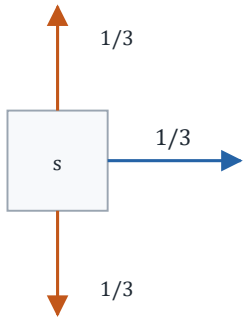
Qué es	Un número que el entorno entrega al pasar de S_t a S_{t+1}. Lleva el subíndice t+1 porque llega después de la acción.
Para qué sirve	Es la única señal de qué es bueno. Define el objetivo del problema: el agente no recibe instrucciones, solo recompensas.
Cómo se obtiene	La devuelve env.step().
Cuánto vale	En FrozenLake, 1 al entrar a la meta y 0 en cualquier otro caso. En CliffWalking, −1 por paso y −100 al entrar al acantilado, que además devuelve al agente al inicio sin terminar el episodio. En Enduro, el puntaje del juego: un punto por cada auto que se adelanta.

Nota 5 $p(s', r \mid s, a)$ dinámica o modelo del entorno

Qué es	La probabilidad de llegar al estado s′ con recompensa r si en el estado s se hace la acción a. La comilla de s′ indica el estado siguiente.
Para qué sirve	Con el modelo se pueden calcular los valores exactos sin interactuar (programación dinámica). Los métodos de los TP no lo necesitan: aprenden de la experiencia.
Cómo se obtiene	Lo conoce quien diseñó el entorno. Gymnasium lo expone en env.unwrapped.P para FrozenLake y CliffWalking; en los TP solo se usó para calcular referencias, nunca dentro de los agentes.
Cuánto vale	Probabilidades entre 0 y 1 que suman 1 para cada par (s, a). FrozenLake es resbaladizo (figura 2). CliffWalking es determinista: cada acción tiene un único resultado con probabilidad 1. En Enduro el modelo es desconocido y estocástico: los otros autos y los cuadros iniciales sin acción cambian en cada partida.



S inicio, F hielo, H agujero, G meta



acción elegida →: se cumple con 1/3 y con 1/3 cada una se desliza a ↑ o a ↓

Figura 2. FrozenLake 4x4 con los números de estado. A la derecha, el efecto de is_slippery: la acción elegida se cumple solo un tercio de las veces.

1.1 Retorno y descuento

El agente no busca la recompensa inmediata sino la suma de las recompensas futuras, descontadas:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \ldots = R_{t+1} + \gamma G_{t+1} \tag{1}$$

Nota 6 G_t retorno

Qué es	La suma descontada de las recompensas que se reciben desde el paso t hasta el final del episodio.
Para qué sirve	Es lo que el agente quiere maximizar. En Monte Carlo es además el dato con el que se aprende (Proceso de decisión de Markov, págs. 29 y 30).
Cómo se obtiene	Solo se conoce cuando el episodio termina. La forma recursiva de la ecuación 1 permite calcularlo recorriendo el episodio de atrás hacia adelante con $G \leftarrow R + \gamma G$ (figura 3), que es lo que hace el bucle hacia atrás del TP1.
Cuánto vale	En FrozenLake con $\gamma = 1$ vale 1 si el episodio llega a la meta y 0 si no; con γ menor que 1, llegar en el paso T vale γ^{T-1}. En CliffWalking con $\gamma = 1$ es menos la cantidad de pasos, menos 100 por cada caída: el mejor posible es −13. En Enduro las curvas del TP3 muestran el retorno sin descontar, es decir, el puntaje del episodio.

Nota 7 γ factor de descuento

Qué es	Un número entre 0 y 1 que multiplica a cada recompensa futura una vez por cada paso de demora.
Para qué sirve	Dos cosas: que la suma sea finita en tareas sin final y que el agente prefiera recibir antes (Proceso de decisión de Markov, págs. 31 y 36).
Cómo se obtiene	Es un hiperparámetro: lo fija quien diseña el agente (GAMMA en el código).
Cuánto vale	TP1 y TP2 usan $\gamma = 1$, posible porque los episodios terminan. TP3 usa 0.99. Una regla útil es el horizonte efectivo $1/(1-\gamma)$: con 0.99 el agente mira unos 100 pasos hacia adelante y con 0.9 unos 10. En el TP1, bajar γ hizo que convinieran caminos más cortos aunque más riesgosos: con $\gamma \leq 0.8$ la política óptima cambia de ruta y su tasa de éxito baja de 74.8% a 44.5%.

Términos G_t 6 | R_{t+1}, r 4 | γ 7 | t, T 1

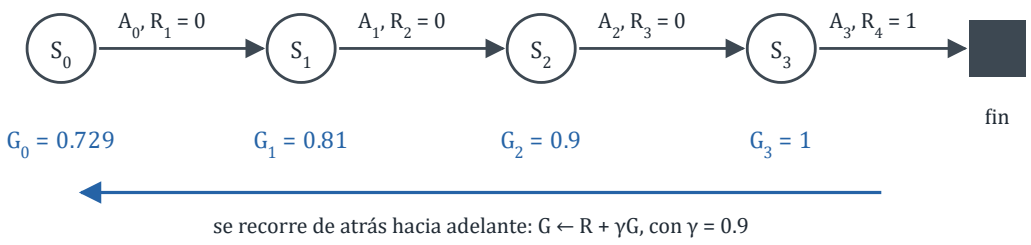


Figura 3. Cálculo del retorno de atrás hacia adelante en un episodio que llega a la meta en el cuarto paso.

1.2 Política y exploración

La política es la regla con la que el agente elige. En los TP siempre se deriva de una tabla o red de valores Q (nota 13): la política greedy elige la acción de mayor valor y la ε greedy agrega exploración.

$$\pi(a \mid s) = 1 - \varepsilon + \varepsilon / |\mathcal{A}| \text{ si } a = \operatorname{argmax}_a Q(s,a); \quad \pi(a \mid s) = \varepsilon / |\mathcal{A}| \text{ en otro caso} \tag{2}$$

Nota 8 $\pi(a \mid s)$ política

Qué es	La probabilidad de elegir la acción a en el estado s. Si una acción tiene probabilidad 1 la política es determinista; si no, es estocástica.
Para qué sirve	Es lo que se quiere aprender en un problema de control: la mejor forma de actuar.
Cómo se obtiene	En los TP no se guarda aparte: se calcula en cada paso a partir de los valores Q (greedy_policy en el TP1, argmax de la salida de la red en el TP3). Cada cambio en Q cambia la política al instante.
Cuánto vale	Probabilidades entre 0 y 1 que suman 1 sobre las acciones de cada estado.

Nota 9 **max, argmax** mejor valor y mejor acción

Qué es	$\max_a Q(s, a)$ es el mayor valor entre las acciones de s; $\operatorname{argmax}_a Q(s, a)$ es la acción que lo alcanza. Actuar greedy es elegir siempre el argmax.
Para qué sirve	Convertir valores en decisiones (argmax) y suponer que en adelante se actuará de la mejor manera (max).
Cómo se obtiene	np.argmax o tensor.argmax sobre la fila de Q del estado.
Cuánto vale	Si hay empates, np.argmax devuelve la primera acción. Al principio, con Q en cero, eso significa ir siempre a la izquierda en FrozenLake y siempre hacia arriba en CliffWalking: un detalle que explica varios resultados de los TP.

Nota 10 **ε** probabilidad de exploración

Qué es	La probabilidad de que la política ε greedy ignore los valores y elija una acción al azar entre todas, incluida la greedy. Por eso la greedy sale con $1 - \varepsilon + \varepsilon/\lvert\mathcal{A}\rvert$ y cada una de las otras con $\varepsilon/\lvert\mathcal{A}\rvert$ (figura 4).
Para qué sirve	Asegurar que se prueben todas las acciones; si nunca se prueba una acción, nunca se sabe cuánto vale (Métodos de Monte Carlo, pág. 53, Métodos de Monte Carlo, págs. 82 a 84). Es el compromiso entre explorar y explotar lo aprendido (Introducción al aprendizaje por refuerzo, pág. 13).
Cómo se obtiene	Hiperparámetro, fijo o con un esquema de decaimiento.
Cuánto vale	Entre 0 y 1. TP1: de 1 a 0.01 con decaimiento geométrico por episodio. TP2: 0.3 fijo en el original y 0.1 en la configuración elegida. TP3: de 1 a 0.01 en forma lineal por paso; 0.01 durante la evaluación.

Términos π(a | s) 8 · ε 10 · A_t, a, $\lvert\mathcal{A}\rvert$ 3 · max, argmax 9

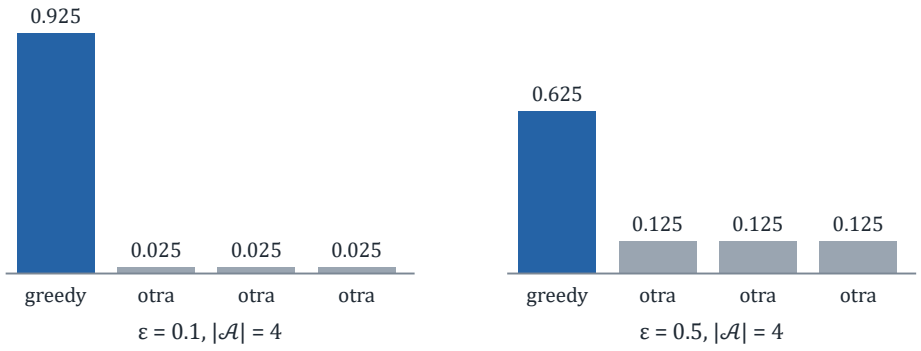


Figura 4. Probabilidades de la política ε greedy con cuatro acciones. Con ε = 0.1 la acción greedy sale el 92.5% de las veces.

1.3 Funciones de valor y ecuaciones de Bellman

Una función de valor resume cuánto retorno se espera desde una situación, siguiendo una política (Proceso de decisión de Markov, págs. 58 y 59):

$$v_\pi(s) = \mathbb{E}_\pi[\, G_t \mid S_t = s \,] \quad q_\pi(s, a) = \mathbb{E}_\pi[\, G_t \mid S_t = s, A_t = a \,] \tag{3}$$

Nota 11 $E_\pi[\cdot]$ esperanza siguiendo π

Qué es	El promedio de una cantidad aleatoria, ponderado por la probabilidad de cada resultado, cuando las acciones salen de π y los estados de la dinámica p.
Para qué sirve	El retorno es aleatorio: cambia de episodio en episodio. El valor es su promedio, un número fijo que se puede comparar.
Cómo se obtiene	Con modelo, como una suma exacta sobre todas las posibilidades. Sin modelo, se estima promediando muestras: eso es lo que hacen Monte Carlo y diferencia temporal.
Cuánto vale	Del mismo orden que la cantidad promediada.

Nota 12 $v_\pi(s), V(s)$ valor de estado

Qué es	El retorno esperado si se arranca en s y se sigue π. Con minúscula, el valor verdadero; con mayúscula, una estimación.
Para qué sirve	Saber qué tan bueno es estar en un estado. Sin modelo no alcanza para decidir, porque no dice adónde lleva cada acción; para eso está q.
Cómo se obtiene	Se relaciona con q promediando sobre la política: $v_\pi(s) = \Sigma_a\, \pi(a \mid s)\, q_\pi(s, a)$. Para una política greedy es el máximo de q.
Cuánto vale	En FrozenLake con $\gamma = 1$ es la probabilidad de llegar a la meta, entre 0 y 1. En CliffWalking es negativo: menos los pasos esperados hasta la meta, contando las caídas.

Nota 13 $q_\pi(s, a), Q(s, a)$ valor de acción

Qué es	El retorno esperado si en s se hace a y después se sigue π. q_π es el valor verdadero, desconocido; Q es la estimación que guarda el agente y que se va corrigiendo con la experiencia.
Para qué sirve	Es lo que permite elegir sin modelo: en cada estado alcanza con comparar las Q de las acciones (Métodos de Monte Carlo, pág. 52). Es el objeto central de los tres TP.
Cómo se obtiene	Arranca en un valor inicial (nota 21) y se actualiza con cada episodio (TP1) o cada paso (TP2 y TP3). En el TP1 y el TP2 es una tabla: un defaultdict o un arreglo con una fila por estado y una columna por acción, 16×4 = 64 y 48×4 = 192 valores. En el TP3 es la salida de una red.
Cuánto vale	En FrozenLake con $\gamma = 1$, entre 0 y 1. En CliffWalking, $Q^*(36, \uparrow) = -13$ y $Q^*(s, \downarrow) = -113$ en los estados junto al acantilado: −100 por caer más −13 desde el inicio. En Enduro, con $\gamma = 0.99$, del orden de los puntos que se esperan en los próximos cien pasos.

Términos $v_\pi(s), V(s)$ 12 · $q_\pi(s, a), Q(s, a)$ 13 · $E_\pi[\cdot]$ 11 · $S_t, s, \mathcal{S}$ 2 · $A_t, a, |\mathcal{A}|$ 3 · G_t 6

La ecuación de Bellman expresa el valor de un par en función de los valores de los pares siguientes. La de optimalidad reemplaza el promedio sobre la política por el máximo (Programación dinámica, págs. 36 a 40):

$$q_\pi(s, a) = \Sigma_{s', r}\, p(s', r \mid s, a)\, [\, r + \gamma\, \Sigma_{a'}\, \pi(a' \mid s')\, q_\pi(s', a')\,] \tag{4}$$

$$q_*(s, a) = \Sigma_{s', r}\, p(s', r \mid s, a)\, [\, r + \gamma\, \max_{a'}\, q_*(s', a')\,] \tag{5}$$

Nota 14 q_*, v_*, π_* valores y política óptimos

Qué es	$q_*(s, a)$ es el mayor valor que puede tener el par (s, a) con cualquier política. La política óptima π_* elige en cada estado el argmax de q_*.
Para qué sirve	Es la meta del control. También sirve de referencia para medir qué tan cerca está lo aprendido.
Cómo se obtiene	Con modelo, por iteración de valor: se aplica la ecuación 5 una y otra vez a todos los estados hasta que los valores dejan de cambiar (Programación dinámica, págs. 71 a 73). En el TP1 y el TP2 se calculó así, solo como referencia.
Cuánto vale	En FrozenLake la política óptima llega a la meta en el 74.8% de los episodios: ningún agente puede superar ese valor. En CliffWalking, $v_*(\text{inicio}) = -13$.

Términos q_*, v_*, π_* 14 | $p(s', r \mid s, a)$ 5 | γ 7 | max, argmax 9

Las ecuaciones 4 y 5 tienen la estructura que se repite en todo el documento: el valor de ahora es la recompensa más γ por el valor de lo que sigue. La programación dinámica la resuelve con el modelo; los métodos de los TP reemplazan la suma sobre p por muestras de experiencia.

2. Monte Carlo off policy con muestreo por importancia (TP1)

2.1 Aprender promediando retornos

Monte Carlo estima los valores promediando los retornos observados en episodios completos. No usa el modelo ni las estimaciones de otros estados: solo lo que realmente pasó (Métodos de Monte Carlo, págs. 9, 28 y 51). El promedio se puede llevar en forma incremental, sin guardar la lista de retornos:

$$Q(s, a) \leftarrow Q(s, a) + (1 / N(s, a)) \cdot [\, G - Q(s, a) \,] \tag{6}$$

Nota 15 $N(s, a)$ cantidad de visitas

Qué es	Cuántos retornos se promediaron para el par (s, a).
Para qué sirve	Con paso 1/N la ecuación 6 da exactamente el promedio de todos los retornos vistos.
Cómo se obtiene	Un contador que suma 1 en cada actualización del par. En el TP1 lo reemplaza C, que suma pesos en lugar de unos (nota 20).
Cuánto vale	Un entero desde 1. Cuanto mayor, menos se mueve Q con cada retorno nuevo.

Nota 16 objetivo objetivo de la actualización

Qué es	El valor hacia el que se corrige la estimación en cada actualización. La diferencia objetivo − estimación es el error.
Para qué sirve	Unifica todos los métodos del documento: estimación ← estimación + paso · (objetivo − estimación). En Monte Carlo el objetivo es el retorno G; en diferencia temporal, R + γ por el valor del estado siguiente (sección 3); en DQN, el valor y calculado con la red objetivo (sección 4).
Cómo se obtiene	Depende del método. Cuanto más depende de muestras aleatorias, más ruido tiene; cuanto más depende de estimaciones, más sesgo puede tener.
Cuánto vale	Del mismo orden que los retornos del problema.

Términos $q_\pi(s, a)$, $Q(s, a)$ 13 · $N(s, a)$ 15 · G_t 6 · objetivo 16

Hay dos variantes. En Monte Carlo de primera visita solo cuenta el retorno desde la primera aparición del par en el episodio; en el de cada visita cuentan todas (Métodos de Monte Carlo, pág. 31). El código del TP1 actualiza en cada visita.

2.2 Control: evaluar, mejorar y explorar

Encontrar la mejor política se llama control. Se usa la iteración de política generalizada: se evalúa la política actual estimando Q con los episodios y se la mejora haciéndola greedy respecto de Q, una y otra vez (Métodos de Monte Carlo, págs. 55, 56 y 61). Para que la evaluación cubra todas las acciones hace falta explorar: con comienzos exploratorios (cada episodio arranca en un par al azar, Métodos de Monte Carlo, pág. 59) o con una política suave como ε greedy (Métodos de Monte Carlo, pág. 82). FrozenLake siempre arranca en el estado 0, así que el TP1 usa ε greedy.

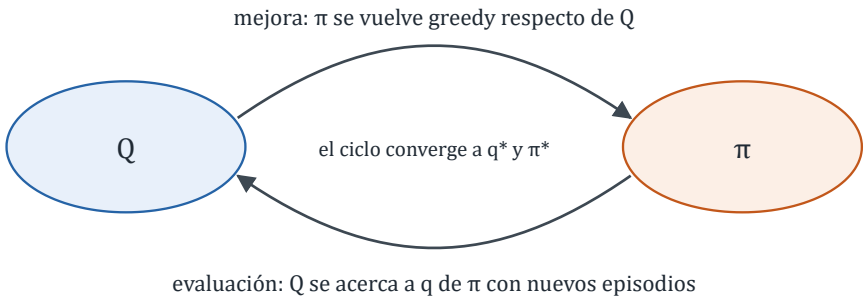


Figura 5. Iteración de política generalizada. En el código la mejora es inmediata, porque la política greedy se lee directamente de Q.

2.3 Dos políticas: objetivo y comportamiento

Un método on policy aprende el valor de la política que ejecuta; uno off policy aprende el de otra (Métodos de Monte Carlo, págs. 81, 86 y 87). El TP1 es off policy: aprende la política greedy π mientras actúa con una política exploratoria μ. La ventaja es aprender la política sin exploración mientras se explora. El problema es que los retornos salen de μ: promediarlos tal cual estimaría el valor de μ, no el de π (Métodos de Monte Carlo, pág. 80).

> Nota 17 $\mu(a \mid s)$ política de comportamiento
>
> | | |
> |---|---|
> | Qué es | La política que realmente genera los episodios. π pasa a llamarse política objetivo: la que se quiere aprender. |
> | Para qué sirve | Explorar sin contaminar lo que se aprende sobre π. |
> | Cómo se obtiene | En el TP1, ε greedy respecto de la misma Q (mu_prob); π es greedy (pi_prob: 1 para la acción greedy y 0 para el resto). En el TP2, Q learning también tiene las dos políticas y SARSA una sola. |
> | Cuánto vale | Las probabilidades de la ecuación 2. Para que el método funcione, μ tiene que poder elegir toda acción que π pueda elegir. |

2.4 Muestreo por importancia

El muestreo por importancia corrige la diferencia pesando cada retorno por cuánto más probable es su trayectoria bajo π que bajo μ (Métodos de Monte Carlo, págs. 89 a 92). La probabilidad de una trayectoria es el producto de las probabilidades de las acciones y de las transiciones; las de las transiciones son las mismas para las dos políticas y se cancelan:

$$\begin{aligned} \rho_{t:T-1} &= \Pi_{k=t}^{T-1} \left[\pi(A_k \mid S_k)\, p(S_{k+1} \mid S_k, A_k) \right] / \left[\mu(A_k \mid S_k)\, p(S_{k+1} \mid S_k, A_k) \right] \\ &= \Pi_{k=t}^{T-1} \pi(A_k \mid S_k) / \mu(A_k \mid S_k) \end{aligned} \tag{7}$$

> Nota 18 ρ razón de muestreo por importancia
>
> | | |
> |---|---|
> | Qué es | El cociente entre la probabilidad de la trayectoria bajo π y bajo μ, que se reduce a un producto de cocientes de políticas. |
> | Para qué sirve | Convertir retornos generados por μ en estimaciones del valor de π, sin conocer el modelo. |
> | Cómo se obtiene | Multiplicando, paso a paso, $\pi(A \mid S) / \mu(A \mid S)$. |
> | Cuánto vale | Cada factor es 0 si π nunca haría esa acción. Con π greedy y μ ε greedy, si la acción fue la greedy el factor es $1/(1 - \varepsilon + \varepsilon/4)$, que va de 1 con ε = 0 a 4 con ε = 1; con ε = 0.1 es 1.081. Al ser un producto, ρ puede crecer de forma exponencial con el largo del episodio. |

Términos ρ 18 · π(a | s) 8 · μ(a | s) 17 · p(s′, r | s, a) 5 · t, T 1

Hay dos formas de usar los pesos:

$$\text{ordinario: } V(s) = \Sigma\, \rho\, G \,/\, N(s) \quad \text{ponderado: } V(s) = \Sigma\, \rho\, G \,/\, \Sigma\, \rho \tag{8}$$

El ordinario no tiene sesgo, pero su varianza puede ser enorme porque ρ no está acotado. El ponderado es un promedio pesado de retornos: tiene algo de sesgo, que desaparece con muchos episodios, y mucha menos varianza. El TP1 usa el ponderado.

2.5 La versión incremental del TP1

El código implementa el muestreo ponderado en forma incremental (Sutton y Barto, sección 5.6). Para cada paso del episodio, de atrás hacia adelante:

$$G \leftarrow \gamma\, G + R_{t+1} \quad C(S_t, A_t) \leftarrow C(S_t, A_t) + W \tag{9}$$

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + (W \,/\, C(S_t, A_t)) \cdot [\, G - Q(S_t, A_t)\,] \tag{10}$$

$$W \leftarrow W \cdot \pi(A_t \mid S_t) \,/\, \mu(A_t \mid S_t); \quad \text{si } W = 0, \text{ cortar} \tag{11}$$

Nota 19 W peso acumulado

Qué es	El valor de ρ para el paso que se está actualizando: el producto de los cocientes de las acciones posteriores.
Para qué sirve	Dar a cada retorno el peso que le corresponde en el promedio ponderado.
Cómo se obtiene	Arranca en 1 al final del episodio y se multiplica por el cociente de cada acción a medida que se recorre hacia atrás. Se multiplica después de actualizar porque el valor de (S_t, A_t) ya supone que se tomó A_t: solo cuentan las acciones siguientes.
Cuánto vale	Con π greedy, W es 0 o una potencia de $1/(1 - \varepsilon + \varepsilon/4)$. En cuanto aparece una acción exploratoria, W se anula y el bucle corta (figura 6). Con π suave nunca se anula pero puede llegar a valores enormes: en la pregunta 2 del TP1 la mediana del W máximo llegó a valores entre 10^{11} y 10^{24}.

Nota 20 $C(s, a)$ suma de pesos

Qué es	La suma de todos los W con que se actualizó el par (s, a). Es el denominador del promedio ponderado.
Para qué sirve	Hace que W / C sea el paso justo para que Q sea exactamente el promedio ponderado de los retornos, el mismo papel que 1/N en la ecuación 6.
Cómo se obtiene	Arranca en 0 (defaultdict con ceros) y suma W en cada actualización del par.
Cuánto vale	Desde 0 hacia arriba, sin tope. Cuanto mayor, menos se mueve Q: los pesos acumulados durante una fase muy exploratoria pueden dominar el promedio y hacer que los episodios posteriores casi no cambien nada (pregunta 7 del TP1).

Términos G_t 6 · C(s, a) 20 · W 19 · $q_\pi(s, a)$, Q(s, a) 13 · ρ 18 · objetivo 16

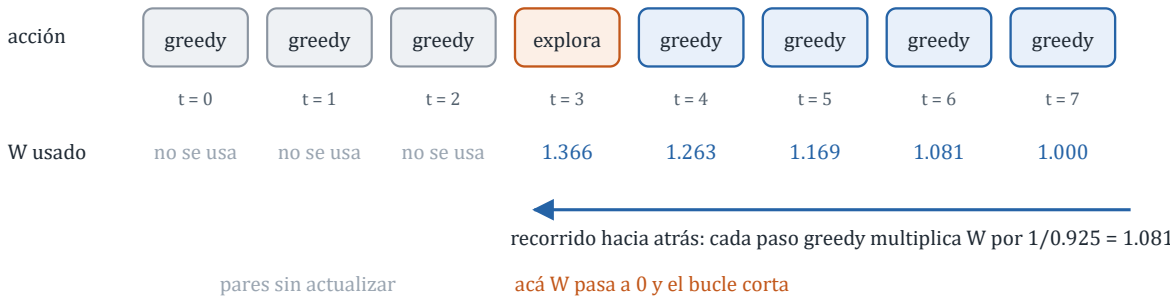


Figura 6. Con π greedy y ε = 0.1, solo se aprende de la cola del episodio posterior a la última acción exploratoria.

De las ecuaciones 9 a 11 salen tres consecuencias que explican casi todas las preguntas del TP1:

1. La primera actualización de un par borra el valor inicial: C pasa de 0 a W, el paso W/C vale 1 y Q queda igual a G (pregunta 1).
2. Con π greedy solo se aprende de la cola de cada episodio, la parte posterior a la última acción exploratoria. Cuanto mayor es ε, más corta es la cola: con ε alto se aprende casi solo del último paso (preguntas 3, 6 y 7).
3. Con π suave W no se anula, pero el producto crece sin control, unos pocos episodios dominan el promedio y la estimación pasa a tener mucha varianza (pregunta 2).

2.6 Valor inicial y decaimiento de ε

Nota 21 Q_0 (Q_INIT) valor inicial de Q

Qué es	El valor que tiene cada Q(s, a) antes de su primera actualización.
Para qué sirve	Las diapositivas lo dejan arbitrario (Métodos de Monte Carlo, pág. 63), pero decide qué hace la política greedy al principio. Si es optimista, es decir mayor que cualquier retorno posible, toda acción probada queda por debajo de las no probadas y la política greedy recorre lo que todavía no probó: una exploración dirigida que se suma a la de ε (Sutton y Barto, sección 2.6).
Cómo se obtiene	Hiperparámetro: el valor con que se crea la tabla (np.zeros o np.full).
Cuánto vale	En FrozenLake es optimista desde 1. En CliffWalking 0 ya es optimista, porque todas las recompensas son negativas: por eso, aun con ε = 0, el agente del TP2 recorre la grilla al principio. En el TP1 su efecto dura poco porque la primera actualización lo borra; en el TP2, con paso α, se va olvidando de a poco (sección 3.2).

GLIE (greedy en el límite con exploración infinita) propone explorar mucho al principio y llevar ε hacia 0, lo que garantiza la convergencia a la política óptima (Métodos de Monte Carlo, pág. 85). El TP1 lo hace con un decaimiento geométrico por episodio:

$$\varepsilon_k = \max(\varepsilon_{min}, \varepsilon_0 \cdot d^k) \tag{12}$$

Nota 22 $d, \varepsilon_0, \varepsilon_{min}, k$ esquema de decaimiento de ε

Qué es	k es el número de episodio, ε_0 el valor inicial (EPSILON_START), ε_{min} el piso (EPSILON_END) y d el factor de decaimiento (EPSILON_DECAY).
Para qué sirve	Pasar gradualmente de explorar a explotar.
Cómo se obtiene	Hiperparámetros. El episodio en que se llega al piso es $\ln(\varepsilon_{min}/\varepsilon_0) / \ln(d)$.
Cuánto vale	Con 1, 0.01 y 0.9995 el piso se alcanza cerca del episodio 9200 (figura 7). Con 0.9999 no se alcanza en 20000 episodios, y en el TP1 eso empeoró el resultado porque con π greedy ε alto acorta las colas que se aprenden.

Términos $d, \varepsilon_0, \varepsilon_{min}, k$ 22 ε 10

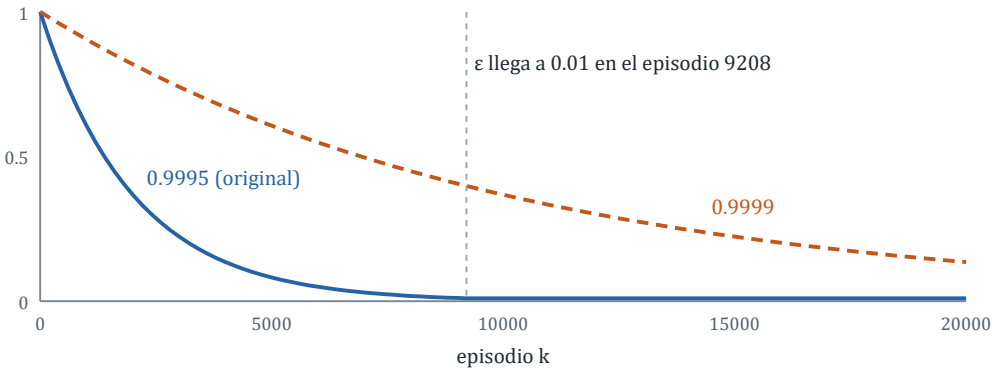


Figura 7. Decaimiento geométrico de ε en el TP1.

3. Diferencia temporal: SARSA y Q learning (TP2)

3.1 Aprender en cada paso

Los métodos de diferencia temporal (TD) aprenden después de cada paso, sin esperar al final del episodio (Métodos de diferencia temporal, págs. 2 y 3). Toman de Monte Carlo el aprender de la experiencia, sin modelo, y de la programación dinámica el bootstrapping: corregir una estimación usando otra estimación, la del estado siguiente. La regla más simple, TD(0), estima V (Métodos de diferencia temporal, págs. 5 y 6):

$$V(S_t) \leftarrow V(S_t) + \alpha\,[\,R_{t+1} + \gamma\,V(S_{t+1}) - V(S_t)\,] \tag{13}$$

Es la misma forma de la ecuación 6 con dos cambios: el objetivo ya no es el retorno completo sino $R + \gamma\,V(S')$, una muestra de la ecuación de Bellman, y el paso ya no es $1/N$ sino una constante α.

Nota 23 α tasa de aprendizaje

Qué es	El tamaño del paso con que se corrige la estimación hacia el objetivo.
Para qué sirve	Decidir cuánta atención se presta a la experiencia más reciente. Un α alto cambia rápido a partir de un solo resultado; uno bajo aprende más lento pero más estable (Métodos de diferencia temporal, pág. 7).
Cómo se obtiene	Hiperparámetro (ALPHA). Con α constante la estimación es un promedio móvil exponencial: el objetivo de hace k actualizaciones pesa $\alpha(1-\alpha)^k$ (figura 8), y el valor inicial pesa $(1-\alpha)^n$ después de n actualizaciones.
Cuánto vale	Entre 0 y 1. TP2: 0.5 en el original y 0.1 en la configuración elegida. Con 0.5, el ruido de cada objetivo mueve mucho la estimación: en el TP2 la política greedy de SARSA quedaba en bucles en 17 de 30 semillas. En el TP3 el equivalente es la tasa de aprendizaje de la red (nota 31).

Nota 24 δ_t error TD

Qué es	La diferencia entre el objetivo y la estimación actual: $\delta_t = R_{t+1} + \gamma\,V(S_{t+1}) - V(S_t)$. Es la cantidad entre corchetes de la ecuación 13.
Para qué sirve	Mide si el paso salió mejor (δ positivo, la estimación sube) o peor (δ negativo, baja) de lo esperado (Métodos de diferencia temporal, págs. 9 y 10). Monte Carlo usa en su lugar el error $G - V$, que necesita el episodio completo (Métodos de diferencia temporal, pág. 11).
Cómo se obtiene	Se calcula en cada transición con la recompensa observada y las estimaciones actuales.
Cuánto vale	Sin cota fija. En CliffWalking una caída produce errores cercanos a −100 en Q(s, ↓). En el TP3 los errores grandes al comienzo motivan la pérdida de Huber (sección 4.4).

Nota 25 S', A' estado y acción siguientes

Qué es	Abreviaturas de S_{t+1} y A_{t+1}.
Para qué sirve	Escribir las reglas de actualización sin subíndices.
Cómo se obtiene	S' lo devuelve el entorno; A' lo elige la política en S'.
Cuánto vale	Los mismos conjuntos que S y A. Si S' es terminal, su valor es 0 por definición y el objetivo es solo R.

Términos $v_\pi(s)$, V(s) 12 · α 23 · δ_t 24 · R_{t+1}, r 4 · γ 7 · S', A' 25 · objetivo 16

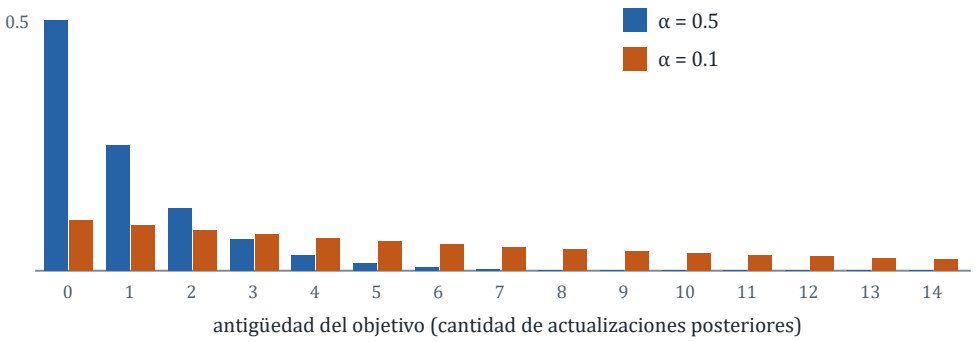


Figura 8. Peso de cada objetivo pasado en la estimación con α constante. Con α = 0.5 casi todo lo deciden los últimos dos o tres objetivos; con α = 0.1 se promedian decenas.

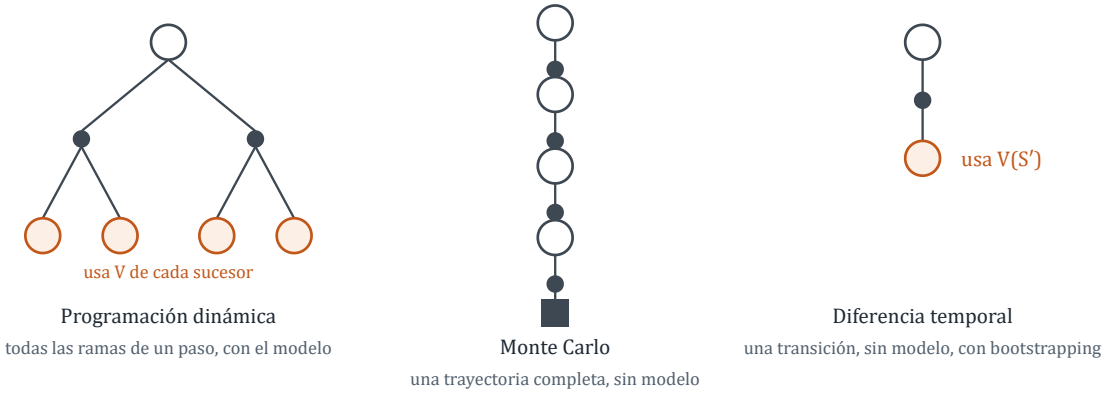


Figura 9. Qué información usa cada familia de métodos para actualizar un valor. Los círculos abiertos son estados, los llenos acciones y el cuadrado un estado terminal.

La diferencia práctica se ve en el gridworld ventoso de las diapositivas: si una mala política deja al agente dando vueltas, Monte Carlo no aprende nada hasta que el episodio termine, mientras que TD corrige en cada paso (Métodos de diferencia temporal, pág. 22).

3.2 SARSA y Q learning

Para controlar sin modelo, TD se aplica a Q en lugar de V (Métodos de diferencia temporal, págs. 12, 13 y 19). Los dos algoritmos del TP2 actúan con la misma política ε greedy y se diferencian en un solo término del objetivo: qué valor del estado siguiente usan (Métodos de diferencia temporal, pág. 30).

$$\text{SARSA:}\quad Q(S,A) \leftarrow Q(S,A) + \alpha\,[\,R + \gamma\,Q(S',A') - Q(S,A)\,] \tag{14}$$

$$\text{Q learning:}\quad Q(S,A) \leftarrow Q(S,A) + \alpha\,[\,R + \gamma\,\max_{a'} Q(S',a') - Q(S,A)\,] \tag{15}$$

Términos $q_\pi(s,a)$, $Q(s,a)$ 13 · α 23 · R_{t+1}, r 4 · γ 7 · S', A' 25 · max, argmax 9

SARSA toma su nombre de la tupla (S, A, R, S′, A′) (Métodos de diferencia temporal, págs. 15 a 18). A′ es la acción que el agente efectivamente va a ejecutar, elegida con ε greedy: SARSA aprende el valor de la política que ejecuta, exploración incluida, y por eso es on policy (Métodos de diferencia temporal, pág. 21). Q learning usa el máximo, es decir, el valor de la acción que tomaría un agente greedy: se comporta ε greedy pero aprende q_*, y es off policy (Watkins, 1989; Métodos de diferencia temporal, págs. 24 a 26 y 41).

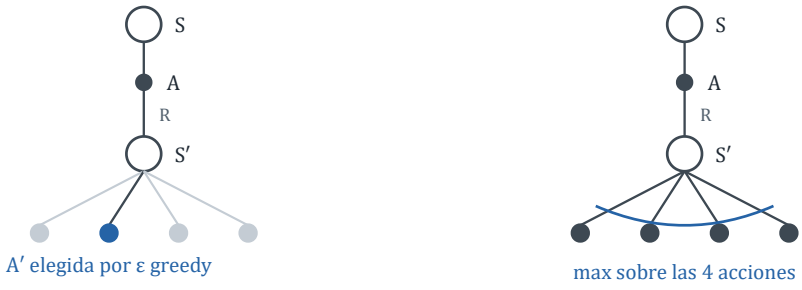


Figura 10. Qué valor del estado siguiente usa cada algoritmo. SARSA usa la acción que eligió; Q learning, la mejor.

Q learning es off policy pero, a diferencia del TP1, no necesita muestreo por importancia. Q(S, A) ya está condicionado a la acción A que se tomó, y el objetivo de la ecuación 15 no depende de qué acción elija después μ, porque toma el máximo. No hay retornos generados por μ que corregir: solo una transición y un máximo sobre estimaciones.

3.3 El acantilado

CliffWalking es una grilla de 4 por 12 con el inicio (36) y la meta (47) en los extremos de la fila de abajo y el acantilado entre ellos (Métodos de diferencia temporal, pág. 36). El camino más corto, de 13 pasos, va pegado al borde (figura 11).

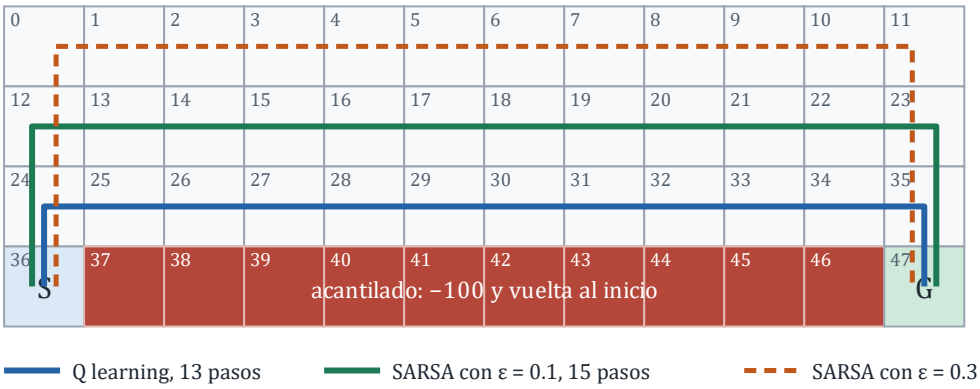


Figura 11. CliffWalking y los caminos greedy aprendidos en el TP2.

1. Q learning aprende los valores de la política greedy, para la cual el borde es el mejor camino: un agente greedy nunca da el paso hacia el acantilado, así que el riesgo aparece solo en Q(s, ↓), cerca de −113, y no en Q(s, →). Pero mientras aprende actúa ε greedy y cae: durante el entrenamiento acumula menos recompensa que SARSA (Métodos de diferencia temporal, págs. 37 y 38).
2. SARSA incluye en Q(S′, A′) los pasos al azar que realmente da. Junto al borde, cada paso tiene probabilidad ε/4 de ser ↓ y costar −100; ese costo entra en el valor de avanzar por la fila 2, Q(s, →) queda por debajo de Q(s, ↑) y prefiere un camino más alejado (Métodos de diferencia temporal, pág. 39). Cuanto mayor es ε, más lejos va: fila 0 con ε = 0.3 y fila 1 con ε = 0.1.
3. Si ε se reduce hacia 0, A′ es casi siempre la acción greedy, Q(S′, A′) se acerca a max Q(S′, ·) y SARSA se parece a Q learning (Métodos de diferencia temporal, pág. 40). Con ε = 0 exacto las ecuaciones 14 y 15 son la misma (pregunta 2 del TP2), y lo que explora es el valor inicial optimista.
4. Al evaluar con algo de exploración residual, la política de SARSA rinde más, porque sus valores ya la tenían en cuenta. El camino de Q learning pasa por 11 estados desde los que una acción al azar lo tira; con ε = 0.05

la probabilidad de caer en una pasada es $1 - (1 - 0.0125)^{11} \approx 0.13$ (pregunta 6 del TP2).

En resumen, SARSA aprende sobre lo que está haciendo y Q learning sobre lo que es mejor (Métodos de diferencia temporal, pág. 42).

Para leer las tablas del TP2 se calcularon con el modelo dos referencias por programación dinámica (Programación dinámica, págs. 71 a 73): Q_*, al que apunta Q learning, y el valor de la mejor política ε greedy, al que apunta SARSA cuando ε queda fijo.

4. Aproximación con redes: DQN y Dueling DQN (TP3)

4.1 Por qué una red

Una tabla guarda un valor por par y aprende cada estado por separado. En Enduro hay demasiados estados para una tabla y casi nunca se repite exactamente la misma imagen. Una red neuronal reemplaza la tabla por una función aproximada Q(s, a; θ) que generaliza entre estados parecidos (Deep Q Network, págs. 2 a 4 y 7). DQN es Q learning con esa red: Mnih y otros (2013) lo propusieron para juegos de Atari, con la imagen del juego como estado y una red convolucional que devuelve un valor por acción (Deep Q Network, págs. 5 y 6).

Nota 26 $\varphi(s)$ estado preprocesado

Qué es	La entrada real de la red: los últimos 4 cuadros del juego, cada uno reducido a 84×84 en escala de grises.
Para qué sirve	Un cuadro solo no muestra hacia dónde se mueven los autos; con cuatro, el estado incluye la velocidad y se acerca a cumplir la propiedad de Markov (nota 2). Reducir la imagen baja el costo.
Cómo se obtiene	AtariPreprocessing (gris, 84×84, cada acción repetida 4 cuadros, hasta 30 cuadros iniciales sin acción) y FrameStack de 4. La red divide por 255 para llevarlo a [0, 1].
Cuánto vale	Un tensor de 4×84×84 = 28224 bytes, con valores enteros de 0 a 255.

Nota 27 θ pesos de la red

Qué es	Todos los parámetros de la red principal: los filtros de las capas convolucionales y las matrices de las capas lineales.
Para qué sirve	Reemplazan a la tabla: Q(s, a; θ) es la salida de la red para la entrada φ(s) en la posición de la acción a.
Cómo se obtiene	Se inicializan al azar (inicialización por defecto de PyTorch) y se ajustan por descenso de gradiente sobre la pérdida (sección 4.2).
Cuánto vale	Números reales, en general chicos. La DQN del TP3 tiene 1 688 745 pesos: 77 984 en el tronco convolucional, 1 606 144 en la capa de 512 y 4 617 en la salida de 9 acciones. Dueling tiene 3 295 402, porque sus dos cabezas duplican la capa de 512 (figura 12).

Términos φ(s) 26 θ 27 $q_\pi(s, a)$, Q(s, a) 13

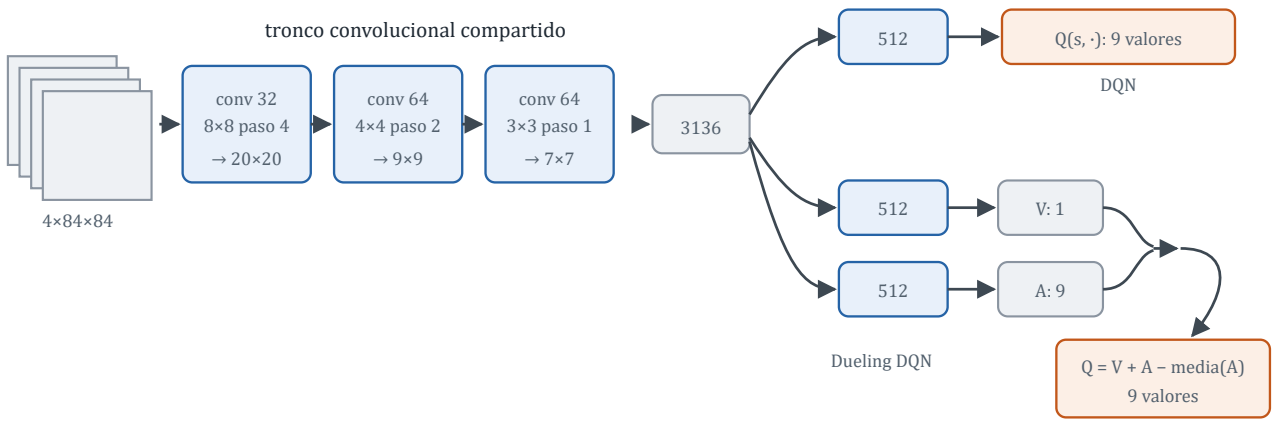


Figura 12. Arquitecturas del TP3. Las dos comparten el tronco convolucional; Dueling separa la salida en V y A (sección 4.6).

4.2 Pérdida, objetivo y gradiente

En vez de actualizar una casilla, se ajustan los pesos para que la predicción se acerque al objetivo de Q learning (Deep Q Network, págs. 8 y 9):

$$y = r + \gamma\,(1 - d)\,\max_{a'} Q(s', a'; \theta^{-}) \tag{16}$$

$$L(\theta) = (1/B)\,\Sigma_{\text{lote}}\,(\,y - Q(s, a; \theta)\,)^2 \quad \theta \leftarrow \theta - \eta\,\nabla_\theta\,L(\theta) \tag{17}$$

Nota 28 d indicador de fin

Qué es	Vale 1 si s' es terminal y 0 si no (en el código, el tensor d de cada transición).
Para qué sirve	Anular el valor del estado siguiente cuando no hay siguiente: con d = 1 el objetivo es solo r, como en las notas 6 y 25.
Cómo se obtiene	Se guarda en el buffer junto con cada transición, a partir de terminated.
Cuánto vale	0 o 1.

Nota 29 y objetivo de DQN

Qué es	El objetivo de Q learning (ecuación 15) calculado con la red objetivo: la recompensa más γ por el mejor valor del estado siguiente.
Para qué sirve	Es el valor hacia el que se empuja la predicción $Q(s, a; \theta)$. $y - Q(s, a; \theta)$ es el error TD δ (nota 24).
Cómo se obtiene	Con la red objetivo y sin gradiente (torch.no_grad): se lo trata como una constante.
Cuánto vale	Del orden de los valores Q. Si la red sobreestima, y también (sección 4.5).

Nota 30 $L(\theta)$, B pérdida y tamaño del lote

Qué es	El promedio, sobre un lote de B transiciones, del error al cuadrado entre objetivo y predicción. B es BATCH_SIZE.
Para qué sirve	Es la cantidad que el descenso de gradiente minimiza. Promediar sobre un lote reduce el ruido de cada paso.
Cómo se obtiene	F.mse_loss en el original y F.smooth_l1_loss (Huber) desde la pregunta 2 del TP3.
Cuánto vale	No negativa. B = 32 en el TP3. Una pérdida que crece sin parar indica divergencia: en la pregunta 1 del TP3, sin red objetivo, llegó a $6.5 \cdot 10^{15}$.

Nota 31 η, ∇_θ tasa de aprendizaje de la red y gradiente

Qué es	$\nabla_\theta L$ es el vector de derivadas de la pérdida respecto de cada peso: indica hacia dónde crece L. η (LR en el código) es el tamaño del paso en sentido contrario.
Para qué sirve	Ajustar los pesos para reducir la pérdida. Cumple el papel de α: la derivada de $(y - Q)^2/2$ es $-\delta\,\nabla Q$, así que el paso es $\theta \leftarrow \theta + \eta\,\delta\,\nabla Q$, la misma forma estimación + paso · error de todo el documento.
Cómo se obtiene	loss.backward() calcula el gradiente y el optimizador Adam da el paso, con un tamaño adaptado a cada peso.
Cuánto vale	LR = 10^{-3} en el original y 10^{-4} desde la pregunta 2 del TP3. Muy alto, los pasos sobrepasan y oscilan; muy bajo, aprende lento.

Términos: y 29 · d 28 · L(θ), B 30 · θ 27 · η, ∇θ 31 · δt 24 · objetivo 16

4.3 Dos problemas y sus soluciones

El primer problema es el blanco móvil. En aprendizaje supervisado el objetivo es fijo; acá y depende de los mismos pesos que se ajustan, y cada paso de gradiente mueve la predicción y también el objetivo. La red persigue una meta que ella misma corre, y aparecen oscilaciones y divergencia (Deep Q Network, pág. 10). La solución de Mnih y otros (2015) es la red objetivo (Deep Q Network, págs. 11 y 12).

> **Nota 32** θ^-, N — red objetivo y período de copia
>
> | | |
> |---|---|
> | Qué es | Una copia de la red principal con pesos θ^- congelados, que se usa solo para calcular y. N es la cantidad de pasos entre copias (TARGET_UPDATE_EVERY; en las diapositivas se llama C, que acá se evita para no confundirla con la suma de pesos del TP1). |
> | Para qué sirve | Darle a la red principal un objetivo fijo durante N pasos, como en un problema supervisado. |
> | Cómo se obtiene | Cada N pasos se copian los pesos: $\theta^- \leftarrow \theta$ (load_state_dict). |
> | Cuánto vale | N = 1 en el original, que equivale a no tener red objetivo; N = 2500 desde la pregunta 1 del TP3. |

El segundo problema es la correlación. Las transiciones consecutivas son casi iguales; entrenar con ellas en orden viola el supuesto de muestras independientes del descenso de gradiente estocástico y la red sobreajusta a lo último que vio (Deep Q Network, pág. 13). La solución es el replay buffer (Deep Q Network, págs. 14 y 15).

> **Nota 33** D, LEARN_START — replay buffer
>
> | | |
> |---|---|
> | Qué es | Una memoria de tamaño fijo con las últimas transiciones (s, a, r, s′, d). Al llenarse descarta las más viejas. |
> | Para qué sirve | Tomar lotes al azar mezcla transiciones de muchos episodios, rompe la correlación y permite reusar cada experiencia varias veces. |
> | Cómo se obtiene | Cada paso del entorno agrega una transición; cada TRAIN_EVERY = 4 pasos se toma un lote de B al azar. LEARN_START es la cantidad de transiciones que se juntan antes de empezar a entrenar, para que los primeros lotes no sean todos iguales. |
> | Cuánto vale | BUFFER_SIZE = 1000 en el original y 50000 desde la pregunta 3 del TP3, que ocupan $50000 \cdot 2 \cdot 28224 = 2\,822\,400\,000$ bytes (2.82 GB) entre s y s′. Con 1000 transiciones un lote viene de uno o dos episodios y la red olvida enseguida lo anterior. |

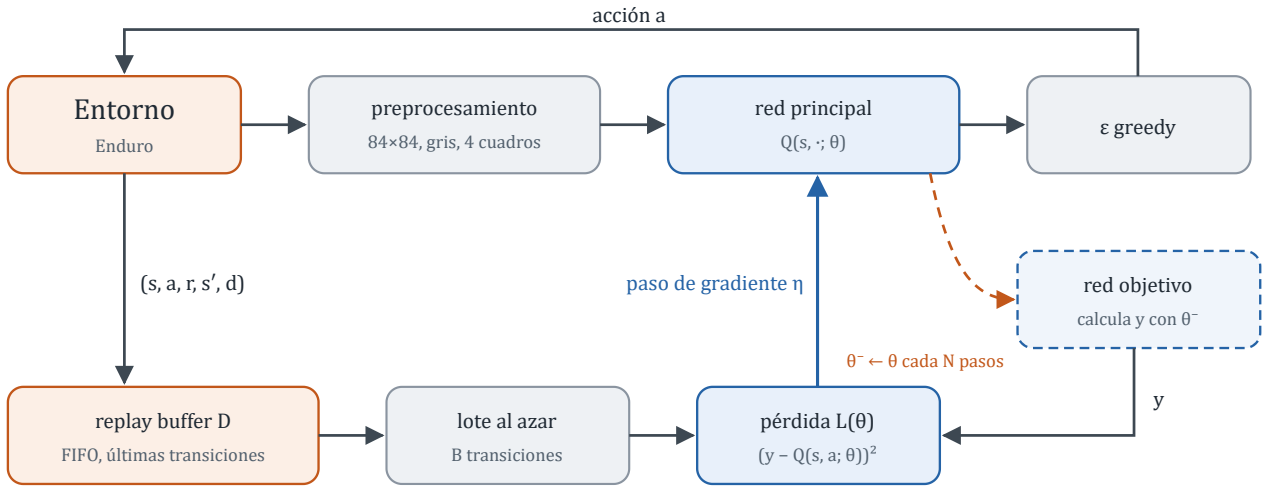


Figura 13. Flujo de datos de DQN con replay buffer y red objetivo.

4.4 Estabilidad del gradiente

Estos recursos no están en las diapositivas; vienen del notebook de clase src/2026/clase_5_dqn y del paper de Mnih y otros (2015). Con la pérdida cuadrática el gradiente es proporcional al error: un solo error enorme,

frecuente al principio, produce un paso enorme que puede arruinar lo aprendido.

$$\text{Huber}(\delta) = \delta^2/2 \text{ si } |\delta| < 1;\ \ |\delta| - 1/2 \text{ si } |\delta| \geq 1 \tag{18}$$

Nota 34 Huber(δ), máximo de la norma — pérdida de Huber y recorte del gradiente

Qué es	La pérdida de Huber es cuadrática para errores chicos y lineal para los grandes. El recorte limita la norma del gradiente total de cada actualización.
Para qué sirve	Acotar cuánto puede mover los pesos una sola muestra o un solo lote, sin cambiar el comportamiento para errores chicos (figura 14).
Cómo se obtiene	F.smooth_l1_loss y nn.utils.clip_grad_norm_(parámetros, 10.0) antes de optimizer.step().
Cuánto vale	Con Huber, la derivada respecto de Q queda entre −1 y 1 para cada muestra; con la cuadrática es 2δ, sin cota. El máximo de la norma es 10 en el TP3.

Términos: Huber(δ), máximo de la norma 34 · δ_t 24 · L(θ), B 30

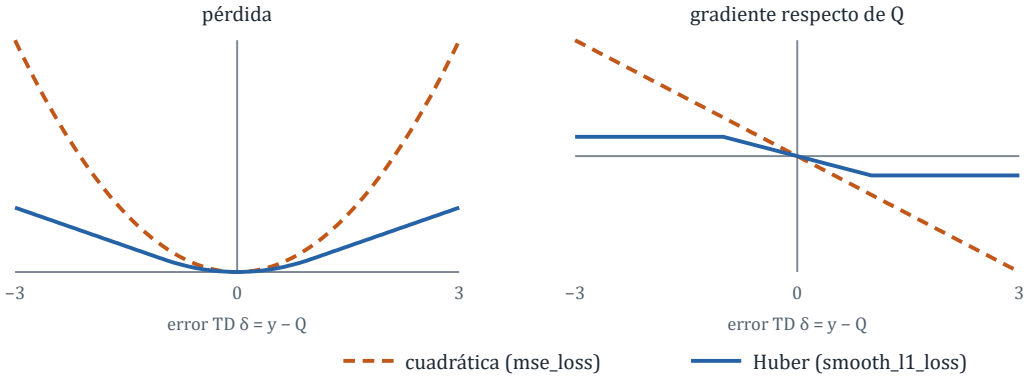


Figura 14. La pérdida de Huber coincide con la cuadrática cerca de 0, pero su gradiente no crece más allá de 1.

La exploración del TP3 usa ε greedy con un decaimiento lineal por paso, la misma idea de GLIE (Métodos de Monte Carlo, pág. 85): ε(paso) = max(ε_{min}, ε_0 − (ε_0 − ε_{min}) · paso / EPSILON_DECAY_STEPS). Si ε llega al mínimo antes de que la red aprenda algo útil, el agente explota estimaciones malas; si llega muy tarde, gasta pasos actuando al azar (pregunta 4 del TP3).

4.5 Sobreestimación y Double DQN

El máximo de la ecuación 16 elige la mejor acción y la evalúa con la misma red. Si una acción está sobreestimada por error, es más probable que la elija; la sobreestimación pasa a y y, por bootstrapping, se propaga a los estados anteriores. El resultado puede ser inestabilidad y políticas peores (Variantes de DQN, págs. 4 y 6). Double DQN separa los dos pasos: la red principal elige y la red objetivo evalúa (Variantes de DQN, págs. 5, 7 y 9). El TP3 no la usa, pero ayuda a leer valores Q que crecen sin control.

$$y^{\text{Double}} = r + \gamma\,(1 - d)\,Q(s', \text{argmax}_{a'}\,Q(s', a'; \theta); \theta^{-}) \tag{19}$$

Términos: y 29 · θ 27 · θ^{-}, N 32 · max, argmax 9

4.6 Dueling DQN

Dueling DQN (Wang y otros, 2016) cambia la arquitectura, no la regla de aprendizaje (Variantes de DQN, págs. 12 a 17). Después del tronco convolucional la red se divide en dos cabezas que se combinan así:

$$Q(s, a; \theta) = V(s) + A(s, a) - (1/|\mathcal{A}|)\, \Sigma_{a'}\, A(s, a') \tag{20}$$

Nota 35 $V(s), A(s, a)$ valor y ventaja en Dueling

Qué es	V(s) estima cuánto vale el estado sin importar la acción, como v (nota 12). A(s, a) es la ventaja: cuánto mejor o peor es cada acción que el promedio. En términos de las funciones de valor, A = q − v.
Para qué sirve	V se actualiza con cada transición, cualquiera sea la acción tomada; en los estados donde la acción casi no importa, la red aprende V sin tener que resolver diferencias chicas entre acciones.
Cómo se obtiene	Dos cabezas lineales sobre el mismo tronco (figura 12). Restar la media fija el reparto: con Q = V + A sin más, sumar una constante a V y restarla a todas las A daría el mismo Q, y el problema sería inidentificable. Con la media, el promedio de Q sobre las acciones es exactamente V (Variantes de DQN, págs. 18 a 21).
Cuánto vale	V del orden de los valores Q. La media de A sobre las acciones es 0 por construcción. En el TP3, la dispersión de las ventajas (máximo menos mínimo) fue en promedio 0.61 y crecía cuando había un auto cerca, como en el ejemplo de Enduro del paper (Variantes de DQN, pág. 22).

Términos $q_\pi(s, a)$, Q(s, a) 13 · V(s), A(s, a) 35 · A_t, a, $|\mathcal{A}|$ 3 · $v_\pi(s)$, V(s) 12

4.7 Comparar con pocas semillas

En RL profundo el resultado de una corrida depende mucho de la semilla: la inicialización de la red, la exploración y el orden de los lotes cambian la curva entera. Lo mismo pasó en los TP tabulares, donde las conclusiones se sacaron con 30 semillas. Para decir que un método es mejor que otro hay que comparar la diferencia de medias con su error estándar.

$$EE = s / \sqrt{n} \tag{21}$$

Nota 36 n, s, EE semillas y error estándar

Qué es	n es la cantidad de corridas independientes (semillas), s el desvío estándar de los resultados entre ellas y EE el error estándar de su media.
Para qué sirve	Distinguir una diferencia real del ruido. Una diferencia menor que unos dos errores estándar no permite concluir nada (Introducción al aprendizaje por refuerzo, págs. 12 a 14).
Cómo se obtiene	Se repite el entrenamiento cambiando solo la semilla de numpy, del entorno, del espacio de acciones y de PyTorch.
Cuánto vale	TP1 y TP2: 30 semillas, más las 3 ejecuciones pedidas. TP3: 3 semillas por configuración y 6 en la pregunta 7, donde Dueling quedó 33.9 puntos adelante en la evaluación sin que ningún test lo distinguiera del ruido.

Términos n, s, EE 36

5. Mapa integrador

La tabla resume cómo cada método completa la forma común estimación ← estimación + paso · (objetivo − estimación).

	Monte Carlo off policy (TP1)	SARSA (TP2)	Q learning (TP2)	DQN y Dueling (TP3)
Qué guarda Q	tabla	tabla	tabla	red con pesos θ
Objetivo	G, pesado por W	R + γ Q(S′, A′)	R + γ max Q(S′, ·)	y con la red objetivo
Paso	W / C	α	α	η, con Adam
Cuándo actualiza	al terminar el episodio	en cada paso	en cada paso	cada 4 pasos, con un lote del buffer
Bootstrapping	no	sí	sí	sí
Política aprendida	greedy (off policy)	ε greedy (on policy)	greedy (off policy)	greedy (off policy)
Corrección off policy	muestreo por importancia	no hace falta	no hace falta (max)	no hace falta (max)
Exploración	ε geométrico por episodio	ε fijo y Q inicial optimista	ε fijo y Q inicial optimista	ε lineal por paso
Riesgo típico	colas cortas, varianza de W	ruido de A′ con α alto	caídas al explorar	blanco móvil, correlación, sobreestimación

Índice de notas

1 t, T paso de tiempo y fin del episodio
2 S_t, s, $\mathcal{S}$ estado
3 A_t, a, $|\mathcal{A}|$ acción
4 R_{t+1}, r recompensa
5 p(s′, r | s, a) dinámica o modelo del entorno
6 G_t retorno
7 γ factor de descuento
8 π(a | s) política
9 max, argmax mejor valor y mejor acción
10 ε probabilidad de exploración
11 $E_\pi[\cdot]$ esperanza siguiendo π
12 $v_\pi(s)$, V(s) valor de estado
13 $q_\pi(s, a)$, Q(s, a) valor de acción
14 q_*, v_*, π_* valores y política óptimos
15 N(s, a) cantidad de visitas
16 objetivo objetivo de la actualización
17 μ(a | s) política de comportamiento
18 ρ razón de muestreo por importancia
19 W peso acumulado
20 C(s, a) suma de pesos
21 Q_0 (Q_INIT) valor inicial de Q
22 d, ε_0, ε_{min}, k esquema de decaimiento de ε
23 α tasa de aprendizaje
24 δ_t error TD
25 S′, A′ estado y acción siguientes
26 φ(s) estado preprocesado
27 θ pesos de la red
28 d indicador de fin
29 y objetivo de DQN
30 L(θ), B pérdida y tamaño del lote
31 η, ∇_θ tasa de aprendizaje de la red y gradiente
32 θ^-, N red objetivo y período de copia
33 D, LEARN_START replay buffer
34 Huber(δ), máximo de la norma pérdida de Huber y recorte del gradiente
35 V(s), A(s, a) valor y ventaja en Dueling
36 n, s, EE semillas y error estándar